



System Programming Using POSIX Threads

The previous articles in this series covered files, processes and inter-process communication. Here, the discussion extends to cover the basics of multi-threading using POSIX functions.

Designing a robust multi-threaded application is quite a vast topic so we will touch upon the basics here.

A thread is an operating system entity which does work on a computer, like that of a process. But it uses less overhead, and it gives you an opportunity to use computing resources more efficiently compared to a traditional process. Some common examples of multi-threaded applications are network servers handling client requests, where individual requests can be handled by a separate thread. There are also applications where one thread is doing computations, while another thread is waiting for some I/O (maybe user input or the arrival of data over a network connection) to be completed.

A properly designed multi-threaded application can run very fast on systems that have multiple CPUs, and can be more responsive with higher throughput. So let us start with the basics of creating a thread by using a POSIX call.

Create a thread using the `pthread_create` function. Its prototype is as follows:

```
int pthread_create(pthread_t * thread, pthread_attr_t * attr,
void * (*start_routine)(void *), void * arg)
```

The first argument is the address of a variable where you store the identifier of the created thread. The second argument can be used to set some of the attributes of the created thread (I am not using this for the example). The starting function

(which the created thread is to execute) is specified by the third argument; the fourth can be used to pass arguments to the function that's to be run in the thread.

Please refer to the code in `lfy_thread_1.c` to understand the use of `pthread_create` and some other calls. You can find the file at http://www.linuxforu.com/article_source_code/oct12/system_programming.zip.

To compile code using `pthread` calls, use the linker flag `-lpthread` (gcc filename -o output -lpthread). The program takes the number of threads to be created by a command-line argument, and creates that many of them. The function `thread_entry` is the entry point for the created threads. Notice that an argument is also passed through the `pthread_create` call.

The purpose of `pthread_join` is to wait for the specific thread, whose identifier is passed as the first argument to this function. It can also fetch the value returned by a specific thread, which can be seen in the code. Note that it is not always necessary to use the `pthread_join` call—I have used it so that readers could learn about it.

Also note the use of `pthread_exit`, which terminates the current thread—but if there are other threads, they can continue. Also, you can return some information to the caller using `pthread_exit`, and that is what I have done in the code. Please note the pointer operations for this.

Do be careful not to use `exit` instead of `pthread_exit`—the first will terminate the entire process immediately, and